

OpenMP offloadingを用いた 並列プログラムの異種 GPU上での評価

東京科学大学 情報理工学院 数理・計算科学系
遠藤研究室 21B30971 吉田和真

背景

世界トップスパコンの搭載GPU:

近年では主流のNVIDIA GPUだけでなく、AMDやIntel社のGPUも使用され始めている

→AMDやNVIDIA GPU上で使用可能なGPU並列化手法OpenMP offloadingに着目

引用: <https://top500.org/lists/top500/2024/11/>

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
5	HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461
6	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu	7,630,848	442.01	537.21	29,899

背景:それぞれの並列化手法の特色

	使用可能なデバイス	使用可能な言語	特徴
OpenMP	マルチコアCPU	FORTRANやC/C++言語	ディレクティブ・ベース
OpenMP offloading	AMD GPU, NVIDIA GPU など	FORTRANやC/C++言語	ディレクティブ・ベース
OpenACC	NVIDIA GPU	FORTRANやC/C++言語	ディレクティブ・ベース
CUDA	NVIDIA GPU	FORTRANやC/C++言語	独自の記述形態

※ディレクティブ・ベース:
既存のコードに指示(ディレクティブ)
文を挿入するだけでプログラムの並
列化が可能な記述方式
プログラミングコストが低いというメ
リットがある

←本研究で使用する並列化手法



OpenMP offloadingは異種
GPU上で使用可能という利
点がある

本研究の目的・成果

目的: ①異種GPU間の性能比較

②CPUとGPU間, 違う並列環境の比較

③CPU, GPU間における性能ポータビリティの有無を確かめる

成果: ①AMD GPU上では実行速度が落ちることがあった

②全体としてCPU並列化手法の方がGPU並列化手法を用いた場合より性能変動が大きかった

③CPU, GPU間で性能ポータビリティが維持されていないケースを発見した

サンプルコード: 1. 行列積演算

このプログラムでは $m \times k$ 行列Aと $k \times n$ 行列Bの積を $m \times n$ 行列Cとして求めている

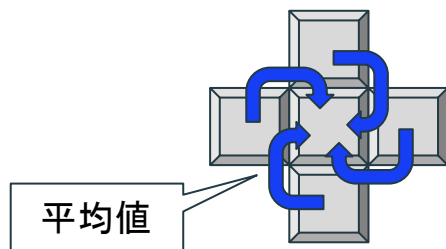
ただし, l, iループは並列化手法によって最適な順番が異なった為, それに合わせた

また, 本研究では同じサイズの正方行列の積を計算する為, $n \times n$ 行列Aと $n \times n$ 行列Bの積を「行列サイズnの行列積」と呼ぶこととする

ソースコード 1: 行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     for (j = 0; j < n; j++) {
5         // 以下のループの順序は並列化手法に
6             よって入れ替える
7         for (l = 0; l < k; l++) {
8             double blj = B[l+j*k];
9             for (i = 0; i < m; i++) {
10                 double ail = A[i+l*m];
11                 C[i+j*m] += ail*blj;
12             }
13         }
14     }
15     return 0;
16 }
```

サンプルコード: 2. diffusion



diffusionプログラムとは、2次元グリッドにおいて、隣り合うグリッドと自身のグリッドの情報の平均値を求め、グリッドの情報を更新していくプログラムである

このプログラムではグリッドサイズを $n_x \times n_y$ 、イテレーション(繰り返し回数)を nt 回としている

また、グリッドの角や辺の情報は固定し、境界における計算は行わない

実験ではイテレーションを1000回に固定

ソースコード 2: diffusion プログラム

```
1 #define IDX(b,x,y) ((b)*(ny)*(nx) + (y)
   *(nx) + (x))
2 void calc(long nx, long ny, long nt)
3 {
4     long t, x, y;
5     for (t = 0; t < nt; t++) {
6         long from = t%2;
7         long to = (t+1)%2;
8         for (y = 1; y < ny-1; y++) {
9             for (x = 1; x < nx-1; x++) {
10                 data[IDX(to,x,y)] = 0.2 * (
                    data[IDX(from,x,y)] + data[
                        IDX(from,x-1,y)] + data[
                            IDX(from,x+1,y)] + data[
                                IDX(from,x,y-1)] + data[
                                    IDX(from,x,y+1)]);
11             }
12         }
13     }
14     return;
15 }
```

サンプルコード:

3. hotspot

ディレクティブが異なる

OpenMP offloadingで読み出される指示文

OpenMPで読み出される指示文

hotspotとは、並列コンピューティングの性能評価を目的とした Rodiniaベンチマークのうち、熱伝導シミュレーションを行うソフトウェアである

1024*1024のグリッドにおいて空間ブロッキングを用いて、時間ステップごとに温度計算を行う(実験ではイテレーションを 10000回に固定)

diffusionと異なるのは、温度計算は平均値ではなく熱抵抗などを用いて行っていること、垂直方面(z軸)からの影響も受けること、角や辺での計算も行っていることである

※ただし、OpenMP offloading用に元のプログラムから改変を行っている (6~10 行目)

ソースコード 3 OpenMP と OpenMP offloading を用いた hotspot プログラム

```
1 void single_iteration(FLOAT *result, FLOAT *temp, FLOAT *power, int
  row, int col, FLOAT Cap_1, FLOAT Rx_1, FLOAT Ry_1, FLOAT Rz_1,
  FLOAT step){
2   FLOAT delta;
3   int r, c, chunk;
4   int num_chunk = row*col / (BLOCK_SIZE_R * BLOCK_SIZE_C);
5   int chunks_in_row = col/BLOCK_SIZE_C;
6   #ifdef OMP_OFFLOAD
7   #pragma omp target teams distribute parallel for ***
8   #else
9   #pragma omp parallel for ***
10  #endif
11  for ( chunk = 0; chunk < num_chunk; ++chunk ){
12    int r_start = BLOCK_SIZE_R*(chunk/chunks_in_row);
13    int c_start = BLOCK_SIZE_C*(chunk%chunks_in_row);
14
15    // 境界条件での計算 (グリッドで角や辺を含むブロックの計算)
16
17    // ***
18
19    // 内部ブロックでの計算
20    for ( r = r_start; r < r_start + BLOCK_SIZE_R; ++r ) {
21      #pragma omp simd
22        for ( c = c_start; c < c_start + BLOCK_SIZE_C; ++c ) {
23          result[r*col+c] = temp[r*col+c] +
24            ( Cap_1 * (power[r*col+c] +
25              (temp[(r+1)*col+c] + temp[(r-1)*col+c]
26                - 2.f*temp[r*col+c]) * Ry_1 +
27              (temp[r*col+c+1] + temp[r*col+c-1]
28                - 2.f*temp[r*col+c]) * Rx_1 +
29              (amb_temp - temp[r*col+c]) * Rz_1));
30        }
31      }
32    }
33  }
```

比較実験内容

実験内容

- ①OpenMP offloadingで実装したプログラムをAMD GPUとNVIDIA GPU上で評価する
- ②OpenMP offloadingの性能をOpenMPで実装した場合と比較する
- ③CPU, GPU間における性能ポータビリティの有無を確かめる

実験環境 ①本研究で使用するマシン

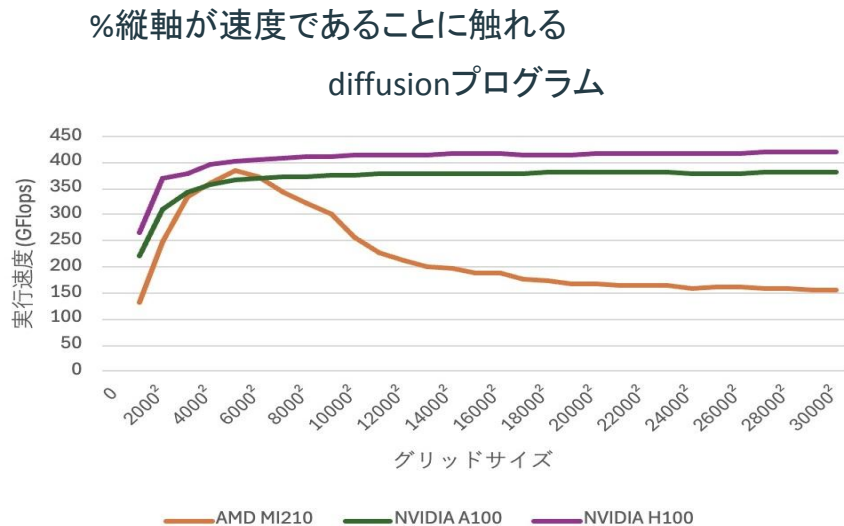
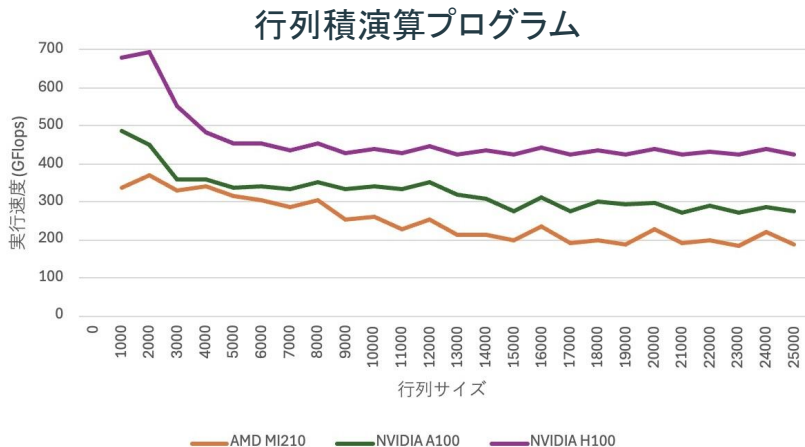
マシン名	CPU	CPUコア数/ スレッド数	GPU	GPUコア(コ ンピュータユ ニット)数	GPU L1キャッ シュ/L2キャッ シュ容量	GPU メモリ容量
マシン1	Intel Xeon Silver 4215 2.5GHz	16コア/ 32スレッド	AMD Instinct MI210	6656	1.625MiB/ 16MiB	64GiB HBM2e
マシン2	AMD EPYC 7702 64-Core Processor	128コア/ 128スレッド	NVIDIA A100 80GB PCIe	6912	20.25MiB/ 40MiB	80 GiB HBM2e
マシン3	AMD EPYC 7773X 64-Core Processor	128コア/ 128スレッド	NVIDIA H100 PCIe	14592	28.5MiB/ 50MiB	80 GiB HBM2e

実験環境 ②本研究で使用するコンパイラ

	OpenMP	OpenMP offloading	OpenACC
マシン1	gcc 8.5.0	clang 17.0.6	-
マシン2	gcc 8.5.0	NVIDIA HPC SDK 21.7	NVIDIA HPC SDK 21.7
マシン3	gcc 8.5.0	NVIDIA HPC SDK 21.7	NVIDIA HPC SDK 21.7

実験結果 1.OpenMP offloadingを用いた性能評価

1-1. 各GPUの実行速度



実験: OpenMP offloadingで実装した行列積演算, diffusionプログラムを各GPUで実行した結果を示す

結果: diffusionプログラムでAMD MI210の実行速度が大幅に低下している

→原因はキャッシュミスであると推測しているが調査中である

(NVIDIA GPUはコアダンプまで実行速度に影響するほどのキャッシュミスを起こさなかった為)

シュ

GPU	GPU L1キャッシュ/L2 キャッシュ容量
AMD MI210	1.625MiB/16MiB
NVIDIA A100	20.25MiB/40MiB
NVIDIA H100	28.5MiB/50MiB

実験結果 2.OpenMPとの比較実験

2-1. 行列積演算

実験: OpenMP, OpenMP offloadingで実装した行列積演算プログラムを3台のマシンで実行した

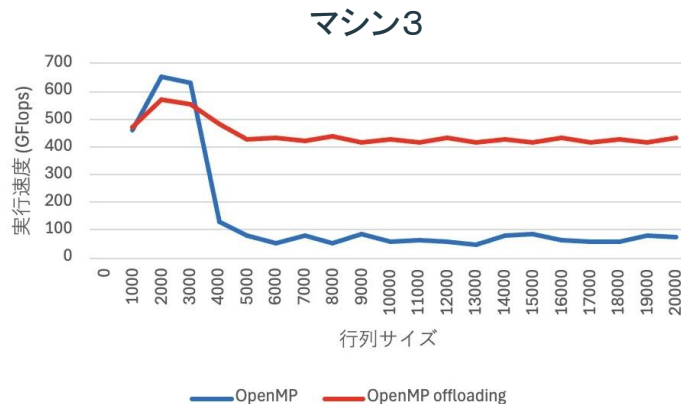
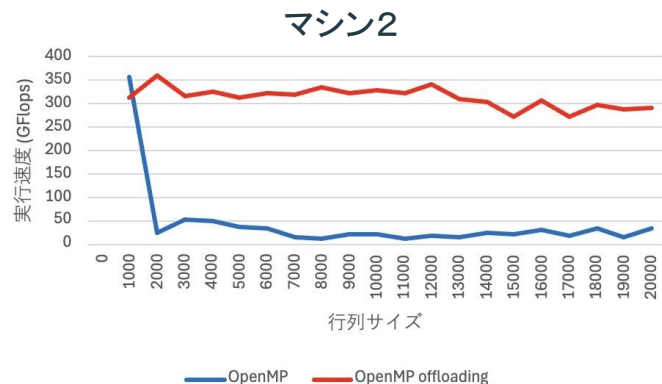
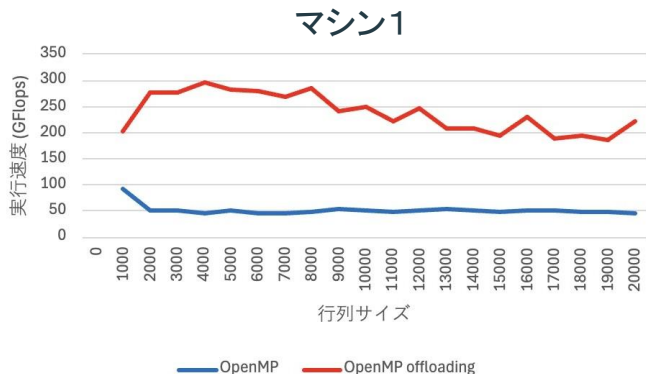
※OpenMP offloadingの実行速度に転送コストを含める

結果: 行列サイズが小さいとOpenMPの方が実行速度が速くなる場合も

OpenMPの性能変動が激しかった

→原因はキャッシュミスにあると考えられる

※転送コスト:
CPU, GPU間でデータの
やり取りをする際の時
間的コスト



実験結果 2.OpenMPとの比較実験

2-2. diffusion

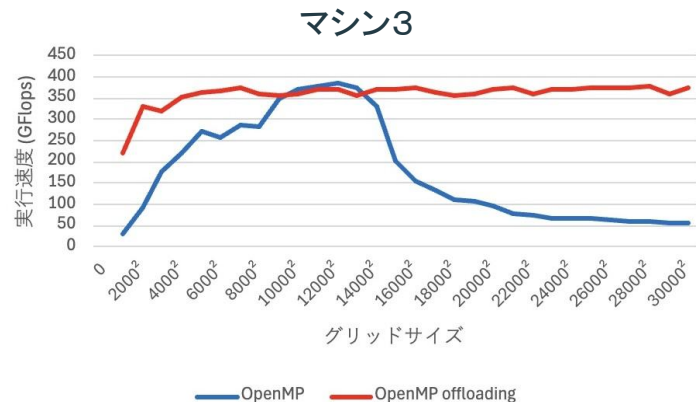
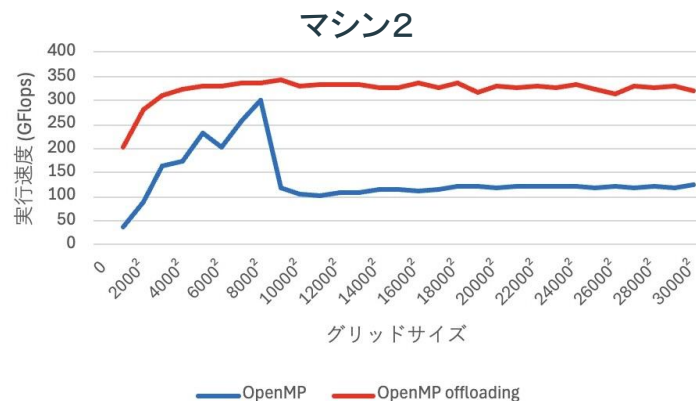
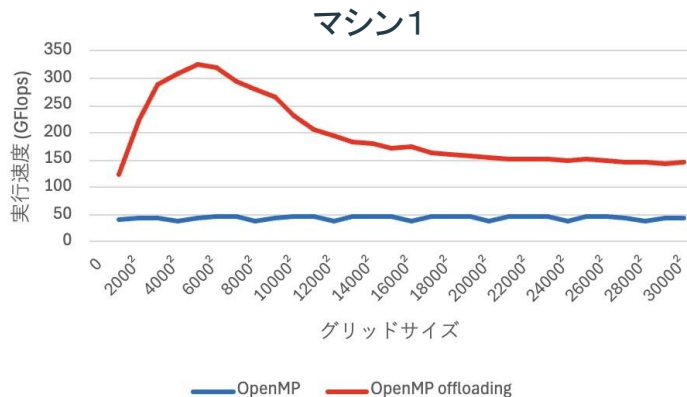
実験: OpenMP, OpenMP offloadingで実装した行列積演算プログラムを3台のマシンで実行した

※OpenMP offloadingの実行速度に転送コストを含める

結果: 行列サイズが小さいとOpenMPの方が実行速度が速くなる場合も

OpenMPの性能変動が激しかった

→原因はキャッシュミスにあると考えられる

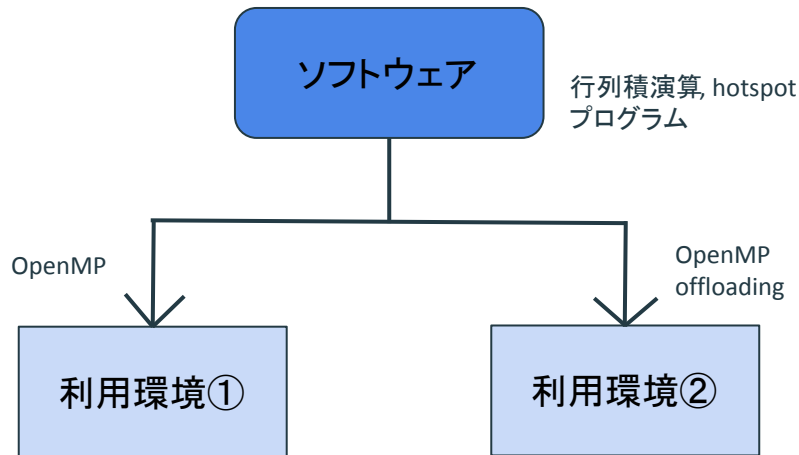


性能ポータビリティとは

性能ポータビリティとは、ソフトウェアを異なる利用環境に移動する時に安定した性能が出せる能力のことを指す。

本研究では、同一のプログラムをOpenMP, OpenMP offloading, OpenACCで実装した場合に性能ポータビリティが保たれているかを評価する。

指示文が違ってても、その他のコードが同一であれば同じソフトウェアであるとする



同様に安定した性能が出せるか

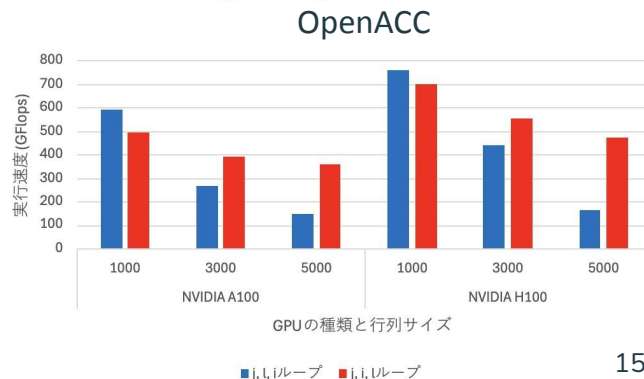
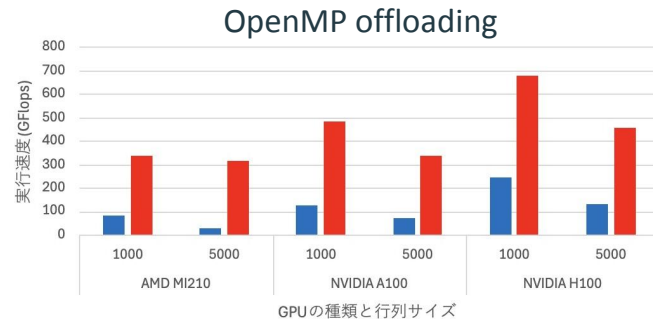
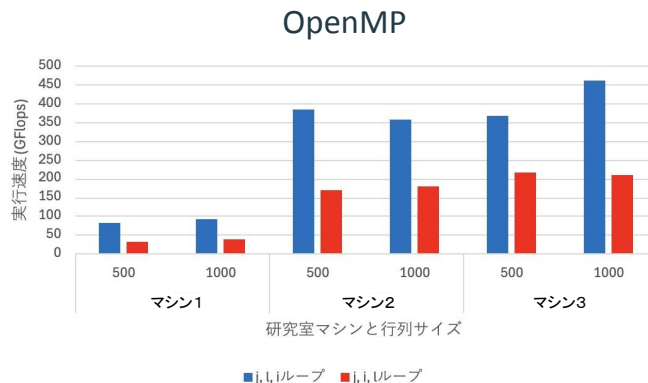
実験結果 3.性能ポータビリティ

3-1. 行列積演算におけるforループ

実験: 行列積演算におけるforループの順番がj, l, iとj, i, lであるプログラムを各並列化手法で実装して 実行速度を測った

結果: OpenMP はj, l, iループが, OpenMP offloadingではj, i, lループのプログラムの方が速くなり, OpenACCは行列サイズによって異なる結果となった

→性能ポータビリティが維持されていない

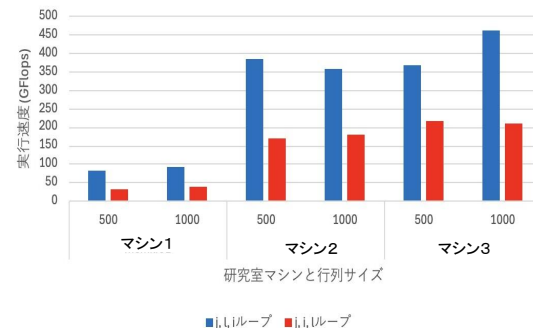


実験結果 3.性能ポータビリティ

3-1. 行列積演算におけるforループ

3-1-1. OpenMP

OpenMPでは, j, l, iループの方が適しているのは、一番内側のループでキャッシュミスが起こりにくいからであると推測される



ソースコード 1 j l i ループ行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     #pragma omp parallel private(i,l)
5     #pragma omp for
6     for (j = 0; j < n; j++) {
7         for (l = 0; l < k; l++) {
8             double blj = B[l+j*k];
9             for (i = 0; i < m; i++) {
10                 double ail = A[i+l*m];
11                 C[i+j*m] += ail*blj;
12             }
13         }
14     }
15     return 0;
16 }
```

配列AもCも次の値を読み込む為、キャッシュミスが起こりにくい

ソースコード 2 j i l ループ行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     #pragma omp parallel private(i,l)
5     #pragma omp for
6     for (j = 0; j < n; j++) {
7         for (i = 0; i < m; i++) {
8             for (l = 0; l < k; l++) {
9                 double ail = A[i+l*m];
10                 double blj = B[l+j*k];
11                 C[i+j*m] += ail*blj;
12             }
13         }
14     }
15     return 0;
16 }
```

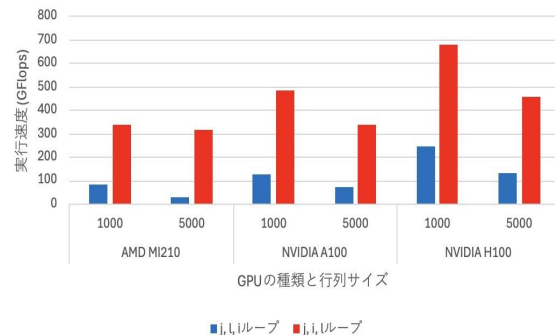
配列Aではm個先の値を読み込む為、キャッシュミスが起こりやすい

実験結果 3.性能ポータビリティ

3-1. 行列積演算におけるforループ

3-1-2. OpenMP offloading

OpenMP offloadingでは, collapse(2)ディレクティブを用いてfor文を結合を行っている為, スレッド数の多いj, lループの方が実行速度が速くなった



キャッシュミスの起こりやすさ < スレッド数の影響力

ソースコード 1 j, lループ行列積演算プログラム

```

1 int matmul()
2 {
3     int i, j, l;
4     #pragma omp target map(to: A[:m*k], B[:k*n], C[:m*n]) map(from: C[:m
      *n])
5     #pragma omp teams distribute parallel for collapse(2)
6     for (j = 0; j < n; j++) {
7         for (l = 0; l < n; l++) {
8             double blj = B[l+j*k];
9             for (i = 0; i < m; i++) {
10                 double ail = A[i+l*m];
11                 C[i+j*m] += ail*blj;
12             }
13         }
14     }
15     return 0;
16 }
```

↑collapse(2)を書くと
衝突が起こる為、不
適切である

ソースコード 2 j, iループ行列積演算プログラム

```

1 int matmul()
2 {
3     int i, j, l;
4     #pragma omp target map(to: A[:m*k], B[:k*n], C[:m*n]) map(from: C[:m
      *n])
5     #pragma omp teams distribute parallel for collapse(2)
6     for (j = 0; j < n; j++) {
7         for (i = 0; i < m; i++) {
8             for (l = 0; l < n; l++) {
9                 double ail = A[i+l*m];
10                 double blj = B[l+j*k];
11                 C[i+j*m] += ail*blj;
12             }
13         }
14     }
15     return 0;
16 }
```

←j, iループ両方を簡
単に並列化できる
n*mスレッドで実行

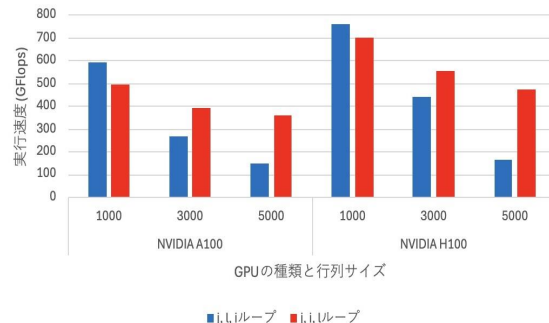
実験結果 3.性能ポータビリティ

3-1. 行列積演算におけるforループ

3-1-3. OpenACC

OpenACCにおいて、行列サイズ1000ではキャッシュミスの起こりにくいj, l, iループが、行列サイズ3000と5000では最初の2つのループが並列化されたj, i, lループが適していると分かった

文法上の自由 :loop independentが離れている2つのfor分で使用可能



ソースコード 1 j l i ループ行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     #pragma acc data copyin(A[0:m*k], B[0:k*n]), copy(C[0:m*n])
5     #pragma acc kernels
6     #pragma acc loop independent
7     for (j = 0; j < n; j++) {
8         #pragma acc loop seq
9         for (l = 0; l < k; l++) {
10             double blj = B[l+j*k];
11         #pragma acc loop independent
12         for (i = 0; i < m; i++) {
13             double ail = A[i+l*m];
14             C[i+j*m] += ail*blj;
15         }
16     }
17 }
18 return 0;
19 }
```

← iループの処理
が1周終わるま
で, lが増えない

ソースコード 2 j i l ループ行列積演算プログラム

```
1 int matmul()
2 {
3     int i, j, l;
4     #pragma acc data copyin(A[0:m*k], B[0:k*n]), copy(C[0:m*n])
5     #pragma acc kernels
6     #pragma acc loop independent
7     for (j = 0; j < n; j++) {
8         #pragma acc loop independent
9         for (i = 0; i < m; i++) {
10             #pragma acc loop seq
11             for (l = 0; l < k; l++) {
12                 double ail = A[i+l*m];
13                 double blj = B[l+j*k];
14                 C[i+j*m] += ail*blj;
15             }
16         }
17     }
18 return 0;
19 }
```

← 最初の2つのループ
を並列化している為
効率良く計算が行わ
れる

実験結果 3.性能ポータビリティ

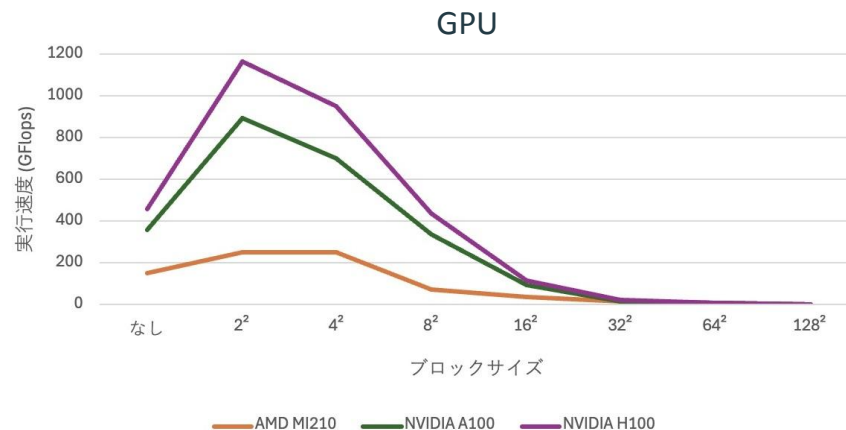
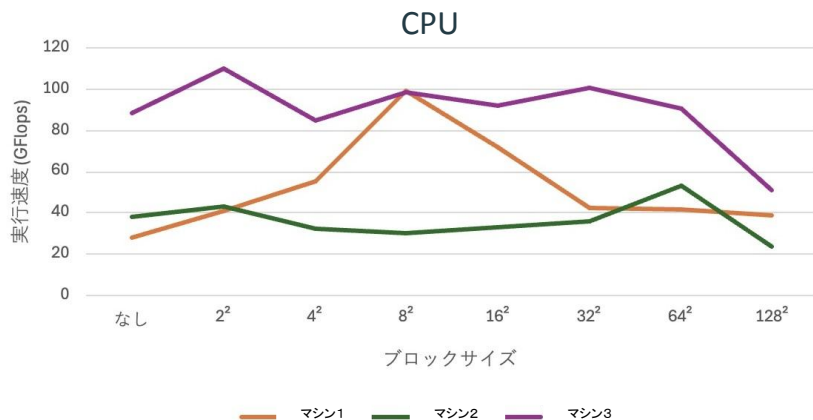
3-2. hotspotにおけるブロックサイズ

実験: OpenMPとOpenMP offloadingで実装したhotspotの実行速度をブロックサイズを変えて測定した

結果: OpenMPではマシンによって適切なブロックサイズが異なる結果となった

OpenMP offloadingではブロックサイズが小さいほど速い結果となった。原因はブロック数に対してGPUのコア数が多い為であると推測される。

→環境ごとにブロックサイズを最適化する必要がある



まとめと今後の課題

まとめ

- ①OpenMP offloadingを使用すると, AMD GPUでの実行速度が極端に下がるケースがある
- ②OpenMPは性能変動が激しい
- ③CPU, GPU間では性能ポータビリティが失われるケースがある

今後の課題

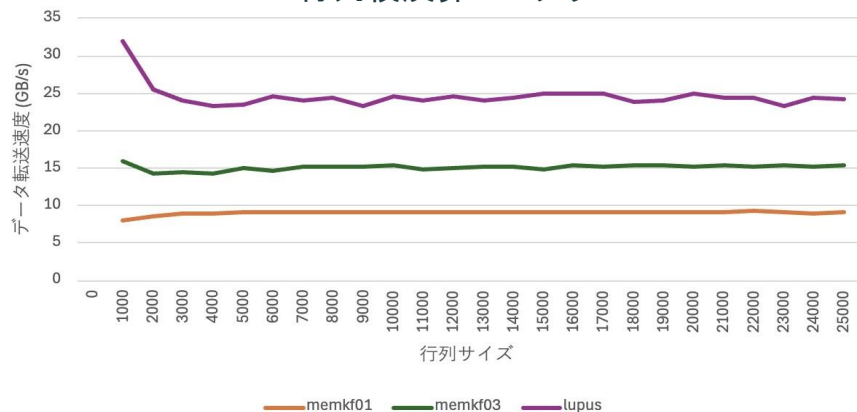
AMD GPUでの速度低下の原因がキャッシュミスにあるのか調べる
様々なプログラムや実行環境において 性能ポータビリティが失われてしまうケースを見つける
性能ポータビリティが保たれるプログラミング方法について研究する
様々なプログラムにおいて適切なブロックサイズとその理由についてPUやGPUのコア数の観点から検証したい

- ✓ 異種GPUを使用することを強調、世界トップスパコンで
NVIDIA以外も使われているから
 - ✓ 転送速度の話を削る
 - ✓ まとめと今後の課題は最悪読み上げなくて良い
- ✓ hotspot14行目から境界条件省く, #pragma .. forのあと...に
する
 - ✓ マシンA, B, Cに変える
 - ・2番でOpenACCを削る, GPU実行速度を省く
 - ・卒論では, 衝突の起きている箇所を直す, collapseを省く
 - ・性能ポータビリティimmのところ, 後日キャッシュミスの回数な
どをプロファイラーで調べる

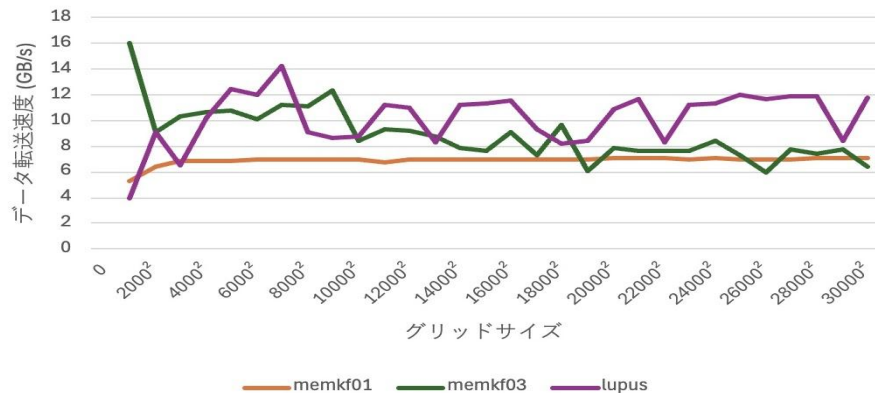
実験結果 1.OpenMP offloadingを用いた性能評価

1-2. 各マシンにおけるデータ転送速度

行列積演算プログラム



diffusionプログラム



実験: OpenMP offloadingで実装した行列積演算, diffusionプログラムを各マシンで実行した結果を示す

結果: diffusionプログラムにおいて, NVIDIA GPUを搭載するmemkf03とlupusの転送速度が不安定になった

→原因は調査中である

背景

ChatGPTを始めとする AI技術の進歩の背景 : 機械学習やディープラーニングの発展

機械学習やディープラーニングの発展の要因

- ・メモリ容量の増加により扱えるデータ量が増えたこと
- ・計算の高速化

本研究ではこちらに注目

→計算の高速化には様々な手法がある

- ・キャッシュ最適化(キャッシュミスを減らす)
- ・ブロッキングの実装

・CPUやGPUによる並列プログラムの実装

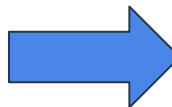
本研究ではこちらに注目

・CPU, GPUのコア数を増やしたり, メインメモリやキャッシュの容量を増やしたりノードを増やしたりするなど, ハードウェアの改良 etc...

→CPU, GPUによる並列化手法

CPU: OpenMP

GPU: CUDA, OpenACC, OpenCL, OpenMP offloading



本研究では異種GPU上で使用可能な
OpenMP offloadingを評価する